

SAL

DDL

→ Grants on the transaction control

## Types of SQL Commands:-

1. DDL: (Data definition language) ⇒ defining relation schema

↳ 1) CREATE: create table, DB, view.

2) ALTER TABLE: modification in table structure  
eg: change col datatype, add/remove column

3) DROP: delete table, DB, view.

4) TRUNCATE: removes all the tuples from the table.

5) RENAME: rename DB name, table name, column name etc.

2. DRL/DQL: (Data retrieval language / data query language)

↳ 1) SELECT: retrieves data from table.

3. DML (Data Manipulation language): use to perform modification in DB.

1) INSERT: insert data into a relation.

2) UPDATE: update relation data.

3) DELETE: delete rows from ~~table~~ relation.

4. DCL (Data Control lang): Grant or revoke authorities from user.

↳ 1) GRANT: access privileges to the DB.

2) REVOKE: revoke user access privileges.

5) TCL (Transaction Control lang): to manage transactions done in the DB.

↳ 1) START TRANSACTION: begin a transaction.

2) COMMIT: apply all the changes and end transaction.

3) ROLLBACK: discard changes and end transaction.

4) SAVEPOINT: checkpoint within the group of transaction in which to rollback.



## Managing DB (DDL):

### 1) Creation of DB:

1) CREATE DATABASE IF NOT EXISTS db name;

2) USE db name;

3) DROP DATABASE IF EXISTS db name;

4) SHOW DATABASES;

5) SHOW TABLES;

### DRL:

1) SELECT < set of col names > FROM < table name >;

2) Order of execution R → L

3) with from clause we can use select by using dual table (dummy tables)

eg: SELECT now();

SELECT user();

SELECT 55+11;

4) WHERE: (reduce rows based on given conditions)

→ SELECT \* FROM Customer WHERE age > 18;

5) BETWEEN:

→ SELECT \* FROM Customer WHERE age between  
(0 & 100 both inclusive) 0 and 100;

6) IN:

→ reduces OR conditions

→ SELECT \* FROM Officers WHERE officername IN

('rahul', 'sai', 'sua', 'Harsha', 'manoj')

7) AND/OR/NOT

8) IS NULL

SELECT \* FROM Customer WHERE prime status is null;

9) Pattern Searching | wildcard ('%', '\_')

→ % ⇒ any no. of chars

→ \_ ⇒ only one char

→ SELECT \* FROM Customer WHERE name LIKE %p\_;

10) ORDER BY:

→ Sorting the data retrieved using WHERE clause.

→ ORDER BY < col name > DESC (or) ASC;

→ eg: SELECT \* FROM Customer ORDER BY name DESC;

11) GROUP BY:

→ GROUP BY clause is used to collect data from multiple records and group the result by one or more column.

→ It is generally used in SELECT statement.

→ Groups into categories based on column given.

→ eg:

SELECT c1, c2, c3 FROM sample table WHERE cond  
GROUP BY c1, c2, c3;

→ In order to successfully execute the query, all the column names mentioned after SELECT statement shall be repeated in GROUP BY.

→ Used with aggregation functions to perform various actions

1) COUNT()

4) MIN()

2) SUM()

5) MAX()

3) AVG()



## 12) DISTINCT:

→ to find the distinct values in the table.

→ `SELECT DISTINCT (colname) FROM table-name;`

→ GROUP BY is also used for the same

1) `SELECT colname FROM table GROUP BY colname;`

Same as above distinct query.

2) SQL is smart enough to realise that if you are using GROUP BY and not using any aggregation function then you mean "DISTINCT"

## 13) GROUP BY HAVING:

(condition)

→ out of the categories made by the group by, we would like to know particular one thing (cond)

→ Similar to WHERE

→ `SELECT COUNT(cust_id), country FROM customer GROUP BY country HAVING COUNT(cust_id) > 50;`

→ WHERE VS HAVING:

→ 1) Both have same fn of filtering the row based on certain condition

→ WHERE: used to filter rows from table on specified condition. HAVING: clause used to filter rows from the groups based on specified condition

→ WHERE is used before GROUP BY, HAVING is used after GROUP BY

→ For using HAVING, GROUP BY is necessary.

## DDL

### → ① PRIMARY KEY:

→ NOT NULL

→ UNIQUE

→ Only one PK

### ② FOREIGN KEY:

→ FK refers to PK of other table.

→ each relation can have any no. of PK's

## 2) CREATE TABLE ORDER

`id INT PRIMARY KEY,`

`delivery-date DATE,`

`order-placed-date DATE,`

`cust_id INT,`

`FOREIGN KEY (cust_id) REFERENCES Customer(id);`

## ③ UNIQUE:

→ Unique can be null, table can have multiple unique attributes

→ eg: `CREATE TABLE customer (`

`email VARCHAR(1024) UNIQUE,`

## ④ CHECK:

→ `CREATE TABLE customer (`

`CONSTRAINT age-check CHECK (age > 12);`

## ⑤ DEFAULT: Set default value of the column.

`CREATE TABLE account (`

`saving-rate DOUBLE NOT NULL DEFAULT 4.25;`

⑥ An attribute can be PK and FK both in a table.



## ④ ALTER OPERATIONS

① changes Schema.

② ADD:-

1. ADD new column:

```
ALTER TABLE tbl-name ADD new_col_name datatype ADD  
new_col_name2 datatype;
```

③ MODIFY: (change data type of an attribute)

```
ALTER TABLE tbl-name MODIFY col_name datatype;
```

④ CHANGE COLUMN:

1. Rename column name:

```
ALTER TABLE tbl-name CHANGE COLUMN old_col_name  
new_col_name new_col datatype;
```

⑤ DROP COLUMN: (drop column completely)

```
ALTER TABLE tbl-name DROP COLUMN col_name;
```

⑥ RENAME: (rename table name itself):

```
ALTER TABLE tbl-name RENAME TO new_table_name;
```

## DML:

① INSERT:

```
INSERT INTO tbl-name (col1, col2, col3) VALUES (v1, v2, v3), (val1, val2,  
val3);
```

② UPDATE:

```
UPDATE tbl-name SET col1=1, col2='abc' WHERE id=1;
```

③ ON UPDATE CASCADE:

If we update PK of the first table then using the ON UPDATE CASCADE FK of the second table automatically get updated.

## ③ DELETE:

1. DELETE FROM tbl-name; // all rows will be deleted;

2. DELETE FROM tbl-name WHERE id=4;

3. ON DELETE CASCADE: on delete PK values in first table  
Second table FK values & also deleted

4. ON DELETE NULL:  
SET

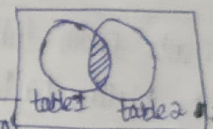
④ REPLACE: [data already present, replace]  
[data not present, insert]  
→ primarily used for already present tuple in a table.  
→ REPLACE INTO table SET col1=val1, col2=val2;

## JOINING TABLES:

1) All RDBMS are relational in nature, we refer to other tables to get meaningful outcomes

2) FK are used to do reference to other table

③ INNER JOIN:



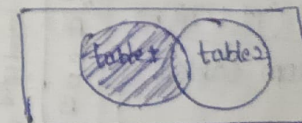
```
SELECT columnlist FROM table1 INNER JOIN  
table2 ON condition1 INNER JOIN table3 ON condition2;
```

④ as (alias)

```
SELECT t1.col1, t2.col2, t1.col3, t2.col4 FROM table1 as  
t1 INNER JOIN table2 as t2;
```

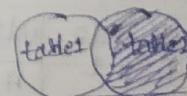
⑤ OUTER JOIN:

1. LEFT JOIN:



```
SELECT cols FROM table1 LEFT JOIN table2 ON join cond;
```

2. RIGHT JOIN:



```
SELECT cols FROM table1 RIGHT JOIN table2 ON join cond;
```



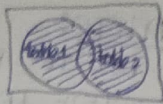
### ③ FULL JOIN: (Implemented in MySQL)

→ LEFT JOIN UNION RIGHT JOIN

→ SELECT cols FROM table1 as t1 LEFT JOIN table2 as t2 ON t1.id = t2.id

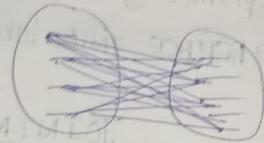
UNION

SELECT cols FROM table1 as t1 RIGHT JOIN table2 as t2 ON t1.id = t2.id;



### CROSS JOIN:

SELECT colists FROM table1 CROSS JOIN table2;



### SELF JOIN:

→ op from particular table when same table is joined itself

SELECT cols FROM table1 as t1 INNER JOIN table1 as t2 ON t1.id = t2.id;

join without using join keyword:

SELECT \* FROM table1, table2 WHERE condition;

### SET OPERATIONS:

→ always gives distinct rows.  
→ Row wise combination.

#### → UNION: (T1 U T2)

SELECT \* FROM table1  
UNION  
SELECT \* FROM table2;

| col1 | col2 |
|------|------|
| A    | 1    |
| B    | 1    |
| C    | 2    |
| B    | 2    |
| D    | 3    |

| col1 | col2 |
|------|------|
| A    | 1    |
| B    | 1    |
| C    | 2    |

| col1 | col2 |
|------|------|
| A    | 1    |
| B    | 2    |
| D    | 3    |



FULL JOIN ⇒

| col1 | col2 | col1 | col2 |
|------|------|------|------|
| A    | 1    | A    | 1    |
| B    | 1    | B    | 2    |
| C    | 2    | NULL | NULL |
| NULL | NULL | D    | 3    |

### ② INTERSECT:

SELECT \* FROM t1  
~~INTERSECT~~  
SELECT \* FROM t2;

→ correlate

SELECT DISTINCT id FROM T1  
INNER JOIN T2 ON  
using USING(id);

### ② MINUS:

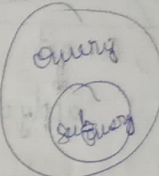
SELECT id FROM T1  
LEFT JOIN T2 USING(id)  
WHERE T2.id IS NULL;

### ③

### SUB QUERIES:

Subqueries mainly exists in 3 clauses:

1. Inside a WHERE clause
2. Inside a FROM clause
3. Inside a SELECT clause



SELECT \* FROM table1 WHERE col1 IN (SELECT col1 FROM table1);

### VIEWS: (duplicate table for original table)

→ CREATE VIEW view-name AS SELECT cols FROM tables [WHERE condition];

→ ALTER VIEW view-name AS SELECT cols FROM table WHERE conditions;

→ DROP VIEW IF EXISTS view-name;



Transactions: represent change in D.B  
 ↳ set of ops used to perform logical unit of work

READ (Access DB)  
 WRITE (Changes in DB)

ACID Properties: (used for doing transactions)

Atomicity  
 either ALL or None

Consistency

Before the trans start & after the trans completed sum of money should be same

Isolation

parallel trans converted into serial trans

Durability

changes in DB, should be must permanent

Rollback, Commit  
 ↓  
 discard changes & trans / apply changes and transaction

ops  
 Rollback → R(A) A=50  
 W(A)  
 R(B)  
 R(C)

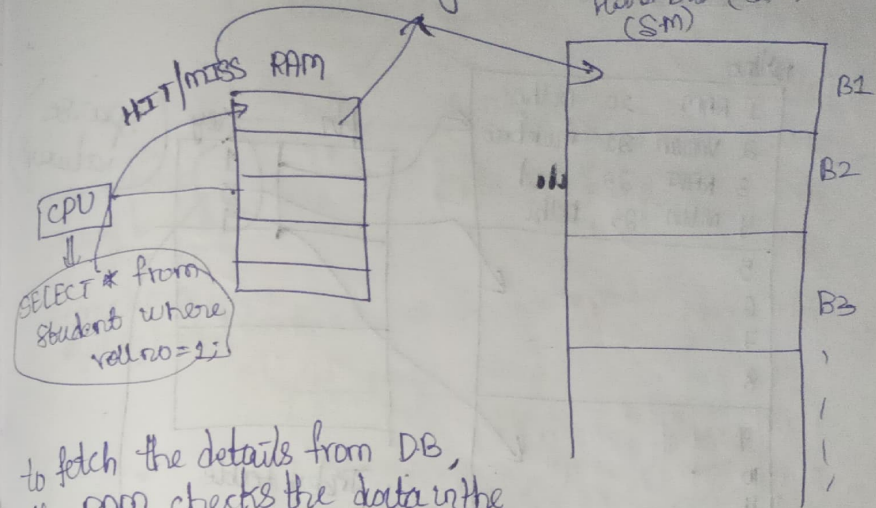
Commit

Before commit any transactions fails then it rolls back for again start from the start  
Atomicity

Consistency:  
Accounts  
 A → 2000  
 B → 3000  
 5000

T1 A - 1000 → B  
 R(A) = 2000  
 A = A - 1000  
 W(A) = 1000  
 R(B) = 3000  
 B = B + 1000  
 W(B) = 4000  
 Commit  
 5000

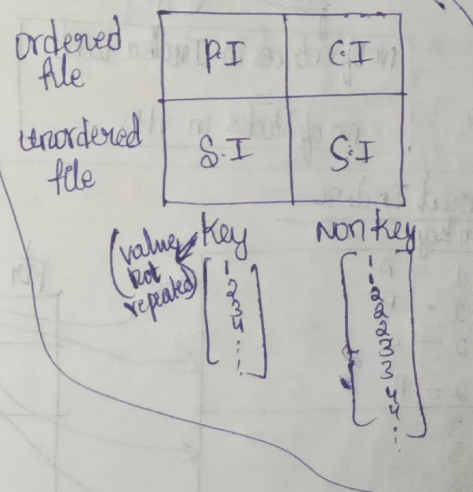
Indexing



to fetch the details from DB, the RAM checks the data in the form of blocks for searching the data in blocks many blocks I/O costs ↑  
 to reduce that no. of blocks calling is reduced by Indexing (I/O cost ↓)

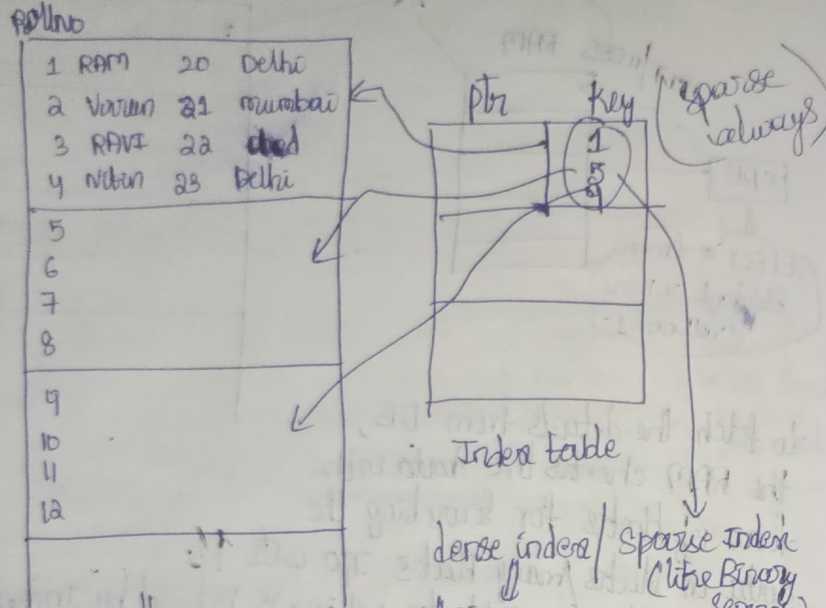
Types of Indexes

- 1) primary (P.I)
- 2) clustered (C.I)
- 3) secondary (S.I)





## Primary (Dynamic Multilevel Index)



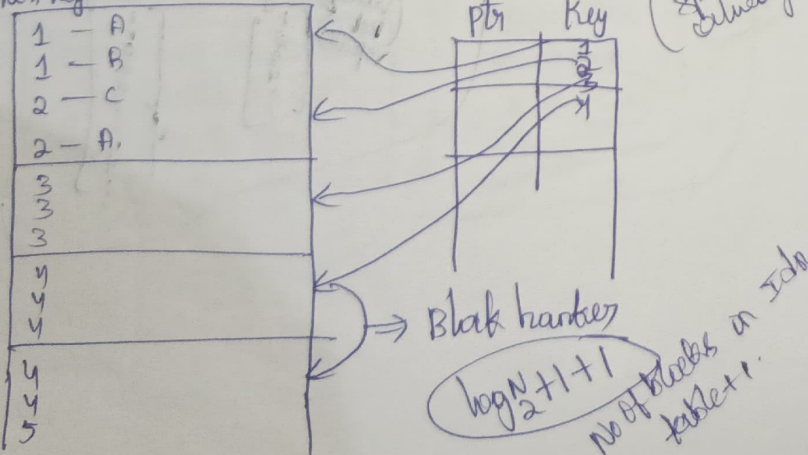
PRIMARY INDEX  $\Rightarrow$  search time  $(\log_2 N + 1)$

no of entries in Index table  
no of blocks in H.D

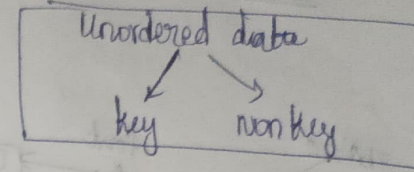
dense index / Sparse Index (like Binary Search)  
Unique indices for every value

## Clustered Index

non key



## Secondary Index



Searching query based on key  $\Rightarrow$  primary Index.

Searching query based on Non key  $\Rightarrow$  Secondary Index (dense)

| eid | name | PAN NO | Key | ptr |
|-----|------|--------|-----|-----|
| 1   | A    | 40     | 23  | 4   |
| 2   | B    | 51     | 33  |     |
| 3   | A    | 62     | 40  |     |
| 4   | C    | 35     | 51  |     |
| 5   | A    | 71     | 62  |     |
| 6   | D    | 82     | 71  |     |
| 7   | E    | 91     | 82  |     |
| 8   | F    | 23     | 91  |     |
| 9   | K    | 100    | 100 |     |
| 10  | L    | 120    |     |     |
| 11  | M    | 150    |     |     |
| 12  | C    | 136    |     |     |
| 13  | AC   |        |     |     |
|     | B    |        |     |     |

Intermediate layer (Block of record ptrs)

| name |
|------|
| A    |
| B    |
| C    |
| D    |